Database Development and Design (SOE201)

Lecture 2a: Indexing Techniques

Dr. Wei Wang
AIAT

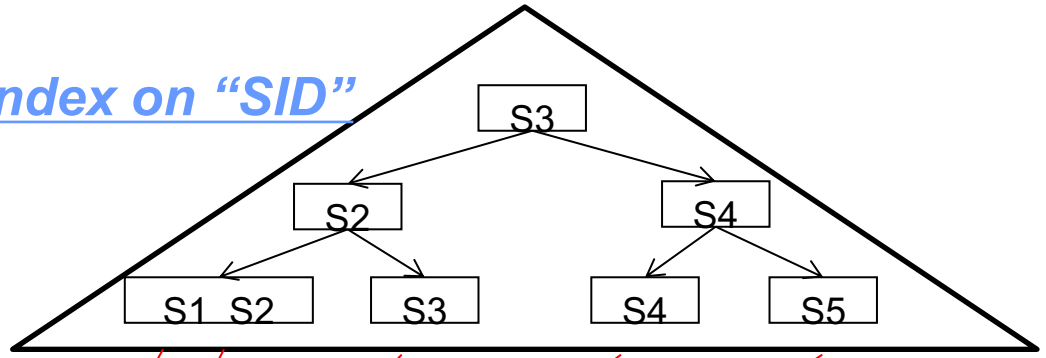
Learning Outcomes

- Structure of Index
- Ordered Index vs. Hash Index
- Primary Index vs. Secondary Index
- Dense Index vs. Sparse Index
- Multilevel index

Motivation: Search Records

- Query issues
  - *SELECT * FROM C*
    - Problem: search all blocks on disk
    - Solution: scattering records of relation on different blocks is inefficient; keep records of relation on adjacent cylinders
  - *SELECT * FROM C WHERE age=10*
    - Problem: search all blocks and check condition
    - Solution: create **indices** on attribute age
- **Indexing** can **speed up** access to desired data.

Index (Indices)



SID	Name	Age	Address
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

Athens	
London	
London	
Paris	
Paris	

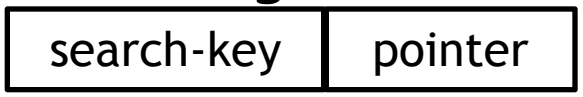
	10
	20
	20
	30
	30

Index on "Address"

Index on "Age"

Structure of Index

- Search Key: one or set of attributes used to look up records.
- Data file: collection of blocks holding records on disk.
- Index file: data structure allowing DBMS to find particular records in data file more efficiently.
  - Index file consists of records (called index entries) of following form:



  - Index files are usually much smaller than original file.

Index Evaluation Metrics

- Access (look up) time
  - Access types supported efficiently
    - Records with specified value on attribute (e.g. name="Leo Messi"), or
    - Records with attribute value falling in specified range (e.g. *10 < age < 20*)
- Insertion time
- Deletion time
- Space overhead

Indexing Techniques

- Index can be:
  - Ordered index where index entries are sorted on search key.
  - Hashing index where hashing technique is employed to organise index entries.

There are many applications using hashing, e.g. password verification, message digest/digital signature, cryptocurrency, database (hash index, hash join)

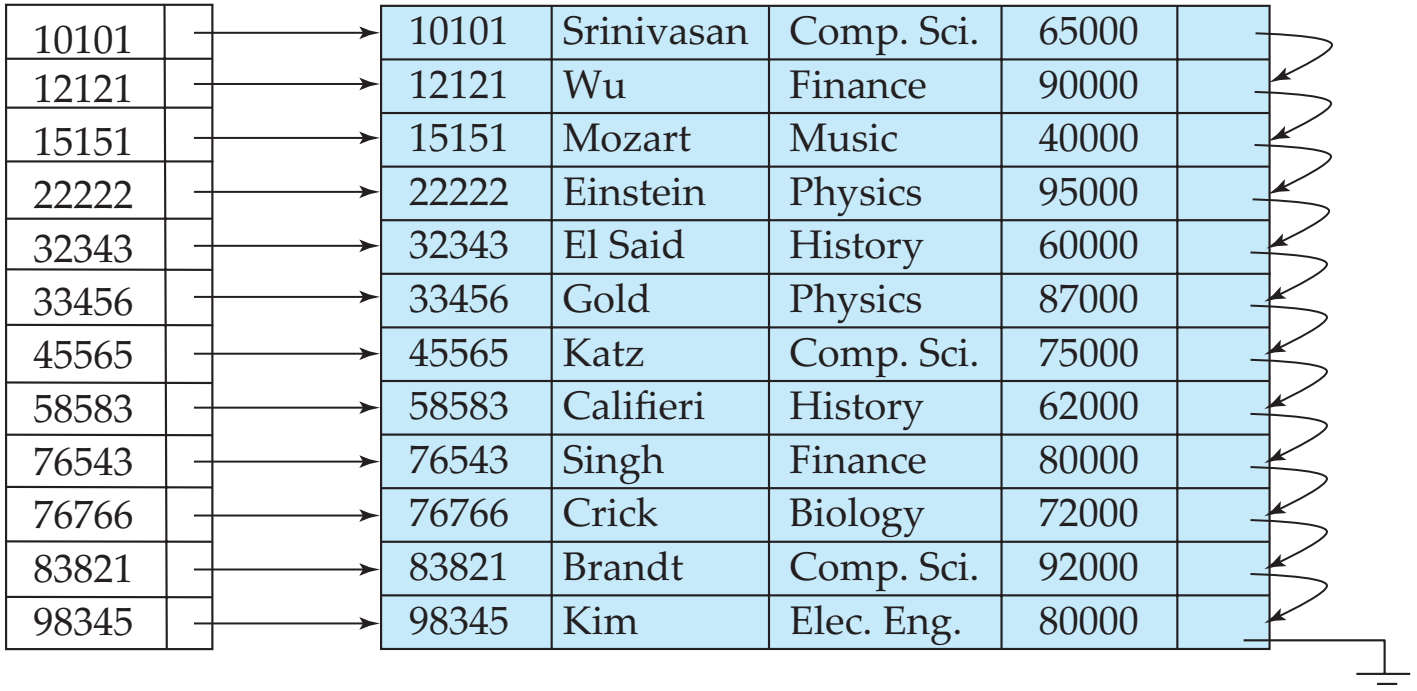
Ordered Index

- Ordered index can be:
  - Dense index: index record appears for every search-key value in file.
  - Sparse index: index record appears for only some search-key values.
    - Typical example is ***one index entry per block.***
    - *This assumption is used throughout this module unless specified explicitly.*

Ordered index: index entries in index are sorted on search key value.

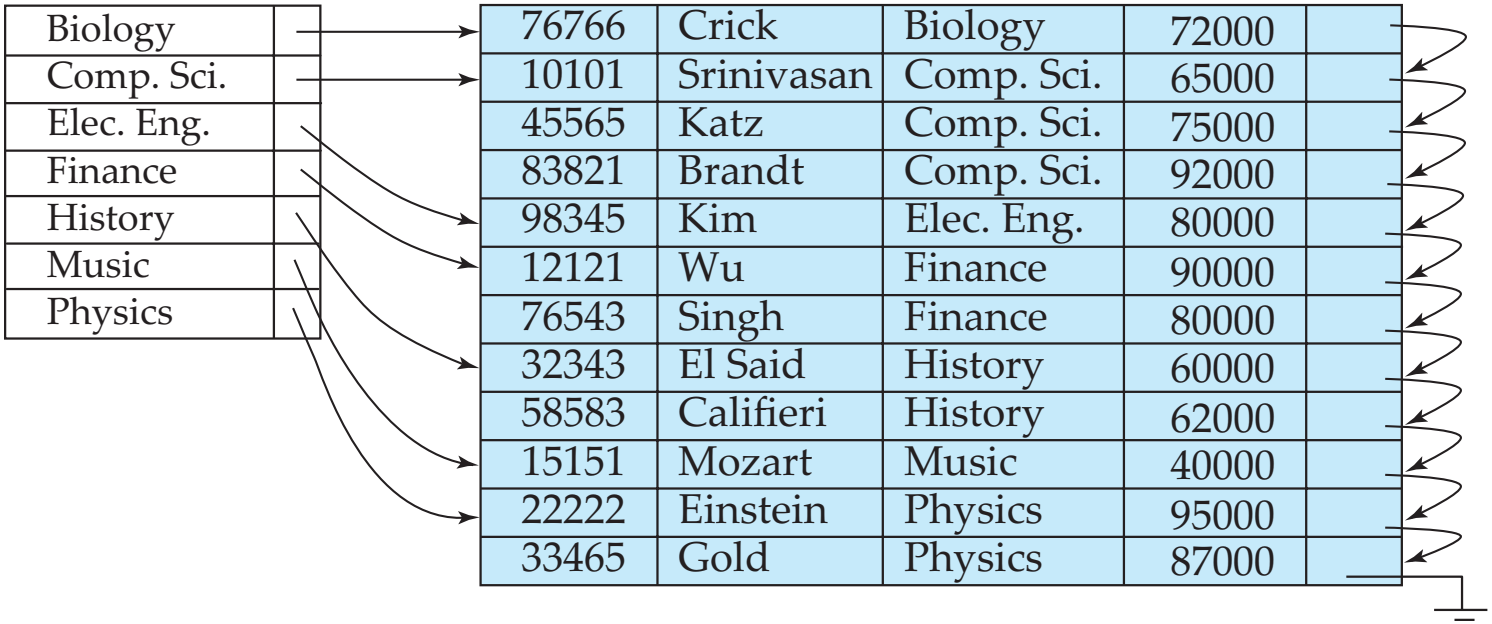
Dense Index

- Dense index - index record appears for every search-key value in file.
  - e.g. index on *ID* attribute of instructor relation



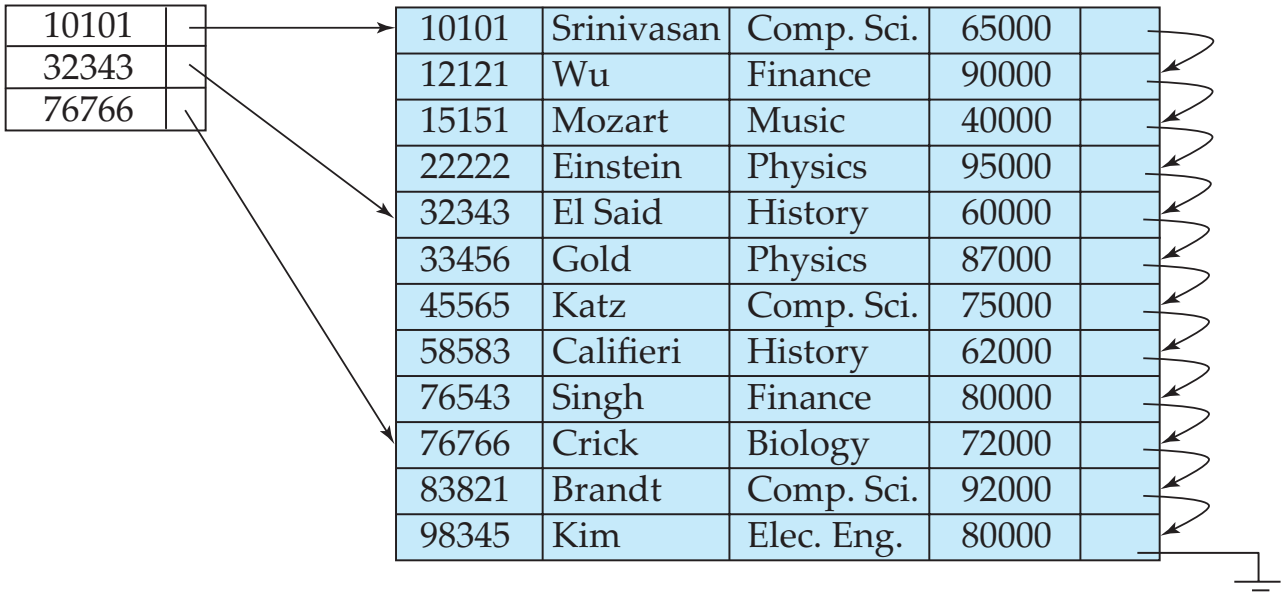
Dense Index – cont'd

- Dense index - index record appears for every search-key value in file.
  - Dense index on *dept_name*, with *instructor* relation sorted on *dept_name*



Sparse Index

- Sparse Index: contains index records for only some search-key values.
  - Applicable only when records are *sequentially ordered* on search-key
- To locate record with search-key value *K*,
  - Find index record with largest search-key *value < K*
  - Search file sequentially starting at tuple to which index record points



Dense Index vs. Sparse Index

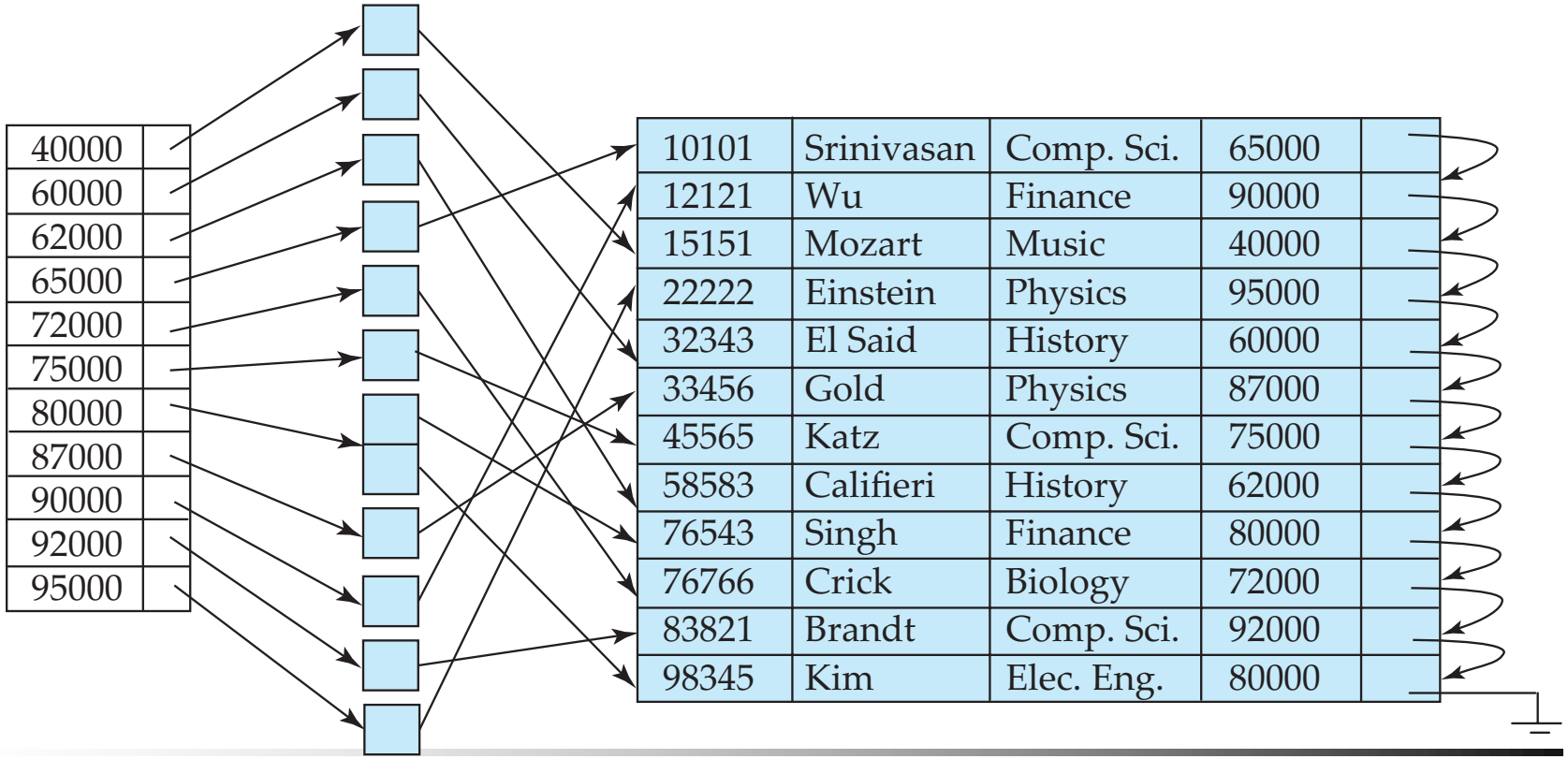
- Index size
  - Sparse index is smaller
- Requirement on data file
  - Data file must be sequential file (sorted) for sparse index
- Lookup
  - Sparse index may fit in memory
  - Dense index can directly tell if record exists
- Update
  - Sparse index requires less space and maintenance for insertion and deletion
- Good tradeoff: sparse index with index entry for every block, corresponding to least search-key value in block.

Ordered Index cont'd

- Ordered index can also be:
  - **Primary** index: index whose search key specifies sequential order of file (file sorted on that key).
    - Also called clustering index (or clustered).
    - Search key of primary index can be but *not necessarily* primary key.
    - Can be sparse
  - **Secondary** index: index whose search key specifies order different from sequential order of file.
    - Also called non-clustering index (or non-clustered).
    - Can *NOT* be sparse

Secondary Index

- Index record points to bucket that contains pointers to all actual records with that particular search-key value.
- Secondary indices have to be dense
  - Example: secondary index on *salary* attribute of instructor relation



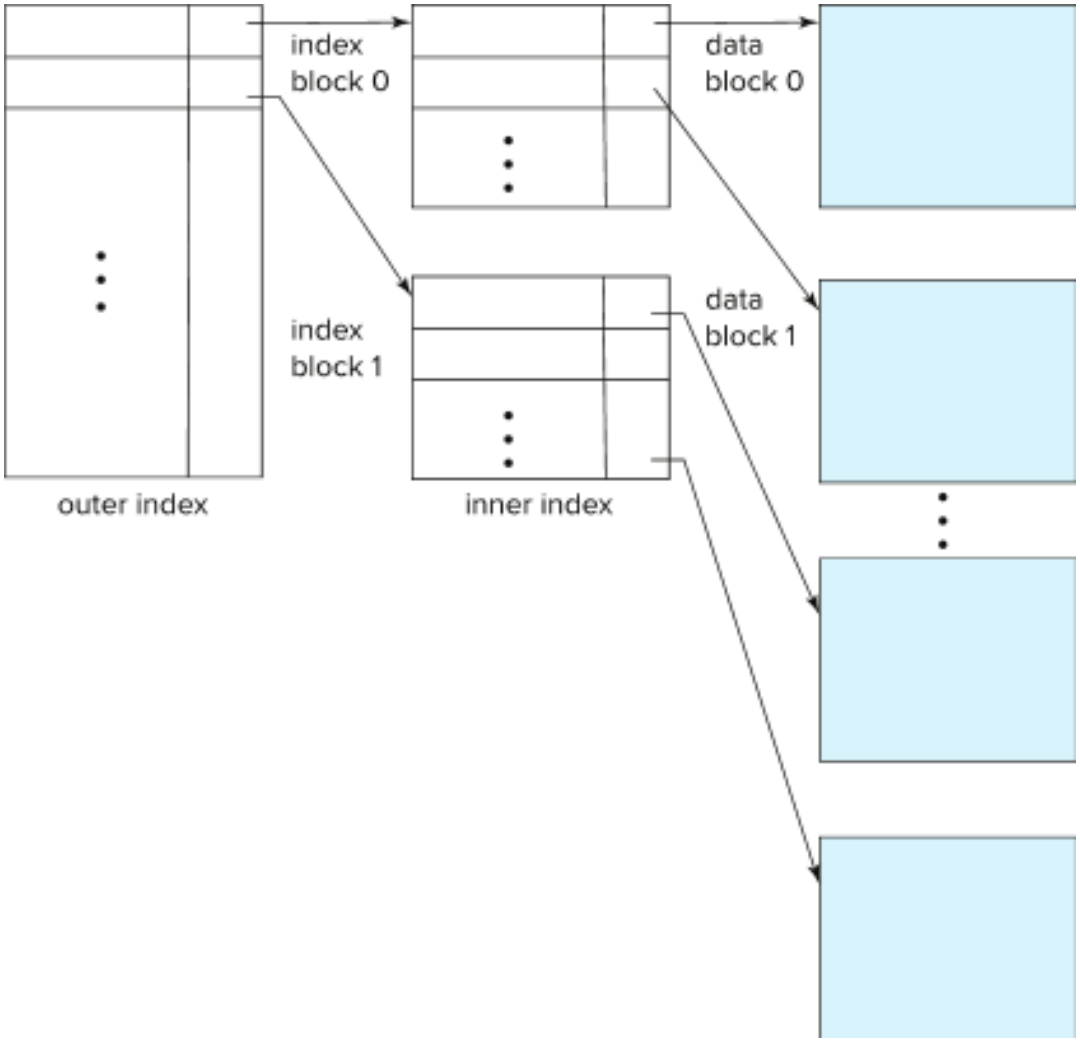
Index Update

- Updating indices imposes overhead on database modification.
  - *When record is inserted or deleted, every index on relation must be updated*
  - *When record is updated, any index on updated attribute must be updated*

Multi-level Index (Index on Index)

- Sequential scan:
  - Using primary index is efficient
  - Using secondary index is expensive
- If primary index does not fit in memory, access becomes expensive.
- Solution: treat primary index kept on disk as sequential file and construct sparse index on it.
  - Outer index – sparse index of primary index
  - Inner index – primary index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on update.

Multi-level Index Example



Index Definition in SQL

- Create index
  - create index <index-name> on <relation-name> (<attribute-list>)
    - e.g. create index b-index on branch(branch_name)
- Drop index
  - drop index <index-name>
    - e.g. drop index b-index

End of Lecture

- Reading
  - Database System Concepts, 7th edition, chapter 14.1, 14.2